

Práctica Tema 3

Serafi Nebot Ginard

01/04/2025

Contents

1	Sistema utilizado	2
2	Implementación del experimento	2
2.1	Monitorización CPU	2
2.2	Monitorización RAM	2
2.3	Sobrecarga de los monitores	2
2.4	Carga	3
2.5	Cálculo número de muestras	3
3	Comportamiento del sistema con diferentes cargas	4
3.1	Utilización de CPU y de RAM	6
3.2	Diferentes utilizaciones de CPU	7
3.3	Comportamiento del sistema con carga fija	8

1 Sistema utilizado

El ordenador en el que he realizado toda la práctica tiene un total de 12 núcleos y 24GB de RAM. He utilizado VMWare Fusion 13.6.3 para crear una máquina virtual con 60GB de disco, 20GB de RAM y 12 núcleos de CPU.

Debido a que me he encontrado con muchos problemas para instalar Ubuntu 16.04, la versión preferible para realizar esta práctica porque se puede instalar una versión de Sysbench < 1.0 directamente, he decidido instalar Ubuntu 24.04.2 y compilar la versión 0.4.12 a partir del código fuente.

2 Implementación del experimento

Para llevar a cabo los experimentos de forma automática he implementado los siguientes apartados, en los que expongo de forma general como he resuelto las diferentes partes del experimento, en un script de Python.

2.1 Monitorización CPU

Para monitorizar el porcentaje de uso de la CPU se ha utilizado el la primera línea del archivo `/proc/stat`:

```
$ cat /proc/stat | head -n 1
cpu 12070 86 2074 516794 445 0 40 0 0 0
```

Que indica la cantidad de *jiffies* de CPU desde que se ha encendido el sistema con el siguiente formato: `cpu [user] [nice] [system] [idle] [iowait] [irq] [softirq] [steal] [guest] [guest_nice]`. Esto significa que la cantidad de jiffies total de CPU es la suma de todos los campos, la cantidad de jiffies que la CPU ha estado sin hacer nada es la suma de los campos `idle` y `iowait`, y la cantidad de jiffies que la CPU ha estado trabajando es la resta entre la cantidad total y la cantidad sin hacer nada.

Debido a que estos campos representan el acumulado hasta el momento, se debe calcular la diferencia de cada uno de los campos para cada intervalo de monitorización. Se puede calcular el porcentaje de utilización total de la CPU:

$$U_{CPU} = \frac{(T_i - T_{i-1}) - (I_i - I_{i-1})}{T_i - T_{i-1}} \cdot 100 = 1 - \frac{I_i - I_{i-1}}{T_i - T_{i-1}} \cdot 100$$

siendo T_i la suma de todos los campos y I_i la suma de los campos `idle` y `iowait`, en el instante i .

2.2 Monitorización RAM

Para monitorizar la utilización de la memoria RAM se han utilizado la primera y tercera línea del archivo `/proc/meminfo`:

```
$ cat /proc/meminfo | head -n 3
MemTotal:      20433748 kB
MemFree:       18669952 kB
MemAvailable:  19156824 kB
```

Que indican la memoria total del sistema y la memoria disponible, respectivamente, en kilobytes ($1KB = 2^{10}B$). Se utiliza `MemAvailable`, la capacidad disponible para utilizar, en lugar de `MemFree`, la capacidad directamente sin utilizar, porque su diferencia con `MemTotal` representa mejor la *cantidad total de memoria utilizada*.

A diferencia del archivo `/proc/stat`, son valores absolutos y no es necesario calcular las diferencias entre intervalos de monitorización. Se puede calcular el porcentaje de utilización directamente:

$$U_{RAM} = \frac{M_{TOTAL} - M_{AVAIL}}{M_{TOTAL}} \cdot 100$$

siendo M_{TOTAL} la memoria total del sistema y M_{AVAIL} la memoria disponible.

2.3 Sobrecarga de los monitores

Para simplificar los datos y los cálculos prefiero tener un intervalo de monitorización, T_I , de 1 segundo, pero necesito saber si la sobrecarga, $O = \frac{T_M}{T_I}$, de los monitores sobre este intervalo de monitorización es aceptable. Para ello he utilizado la

función `time()` del módulo `time` de Python para calcular el tiempo total de monitorización para cada iteración. Ejecutando los monitores a lo largo de unas 10 iteraciones he obtenido los siguientes tiempos:

0.000273, 0.000196, 0.000138, 0.000257, 0.000193, 0.000244, 0.000197, 0.000136, 0.000281, 0.000220

Calculando la media aritmética de todos estos valores obtenemos un tiempo de monitor de $T_M = 0.0002135s$, a partir del cual podemos obtener el overhead:

$$O = \frac{T_M}{T_I} = \frac{0.0002135}{1} = 0.0002135 \Rightarrow 0.02135\%$$

Un overhead de 0.02135% es bastante razonable, por lo que he decidido mantener un intervalo de monitorización de 1 segundo.

2.4 Carga

Para aplicar una carga al sistema se ha utilizado el test de CPU de cálculo de números primos del Sysbench. Por ejemplo, para cargar la CPU al 75% ($12 \cdot 0.75 = 9$ núcleos) con el test de 100000 números primos:

```
$ sysbench --num-threads=9 --test=cpu --cpu-max-prime=100000 run
```

```
sysbench 0.4.12: multi-threaded system evaluation benchmark
```

Running the test with following options:

Number of threads: 9

Doing CPU performance benchmark

Threads started!

Done.

Maximum prime number checked in CPU test: 100000

Test execution summary:

```
total time:                2.0681s
total number of events:    10000
total time taken by event execution: 18.5503
per-request statistics:
  min:                    1.60ms
  avg:                    1.86ms
  max:                    10.75ms
  approx. 95 percentile:  2.76ms
```

Threads fairness:

```
events (avg/stddev):      1111.1111/16.87
execution time (avg/stddev): 2.0611/0.00
```

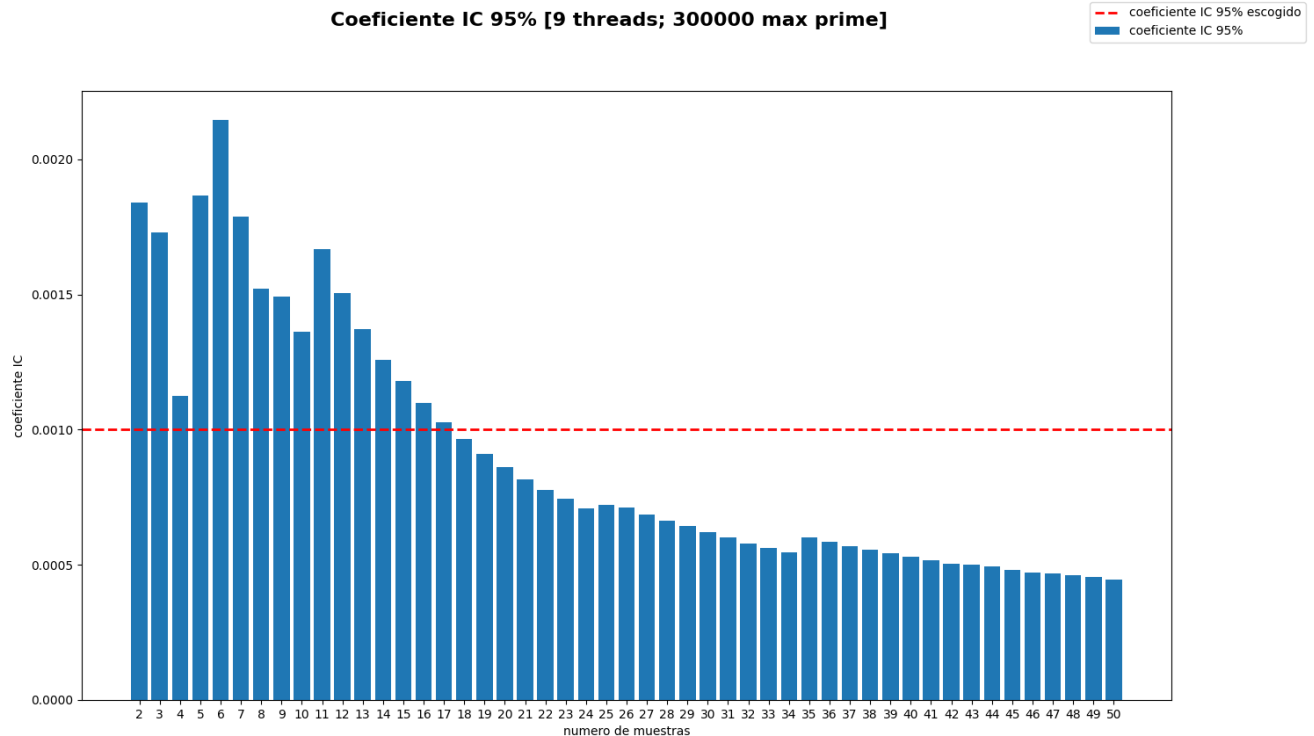
Como el Sysbench ya nos proporciona el tiempo de respuesta (campo `total time`; primera línea del resumen de ejecución) no se necesita monitorizar el tiempo de ejecución del Sysbench.

En concreto, desde Python se crea un subproceso para el Sysbench, se monitoriza la CPU y la RAM (leyendo el fichero correspondiente directamente desde Python) cada segundo hasta que el subproceso termina, se captura la salida estándar y se obtiene el campo `total time` con expresiones regulares.

2.5 Cálculo número de muestras

Para cada carga se han realizado múltiples ejecuciones y se ha calculado la media del tiempo de respuesta. Para estar seguros de que la media que obtenemos es “fiable”, lo que se ha hecho es seguir haciendo ejecuciones hasta llegar a un coeficiente del intervalo de confianza del 95%, $\frac{IC}{R}$, lo suficientemente pequeño.

Para determinar este coeficiente se ha ejecutado una carga de 300000 números primos unas 50 veces y se ha calculado el intervalo de confianza del 95% a partir de un conjunto de 2 muestras hasta llegar al conjunto completo de 50 muestras. La siguiente gráfica muestra el resultado de este proceso.



El coeficiente varía bastante con pocas muestras pero tiende a bajar y estabilizarse a medida que se va aumentando el número de muestras. He elegido un coeficiente de 0.001 (línea discontinua roja) porque he considerado que era un buen punto para separar las muestras iniciales, que son más caóticas, de las muestras ya estabilizadas. Además representa una variación del 0.1% sobre la media, que es un rango muy estrecho.

Es interesante comentar que en las primeras pruebas que he realizado el coeficiente era más alto (0.005 el más pequeño dentro de un conjunto de 50 muestras) y mucho menos estable. Al parecer, este comportamiento estaba ocasionado por los cálculos realizados entre las baterías de 3 ejecuciones. Tras poner una espera de unos segundos antes de volver a realizar 3 ejecuciones más este comportamiento ha desaparecido.

3 Comportamiento del sistema con diferentes cargas

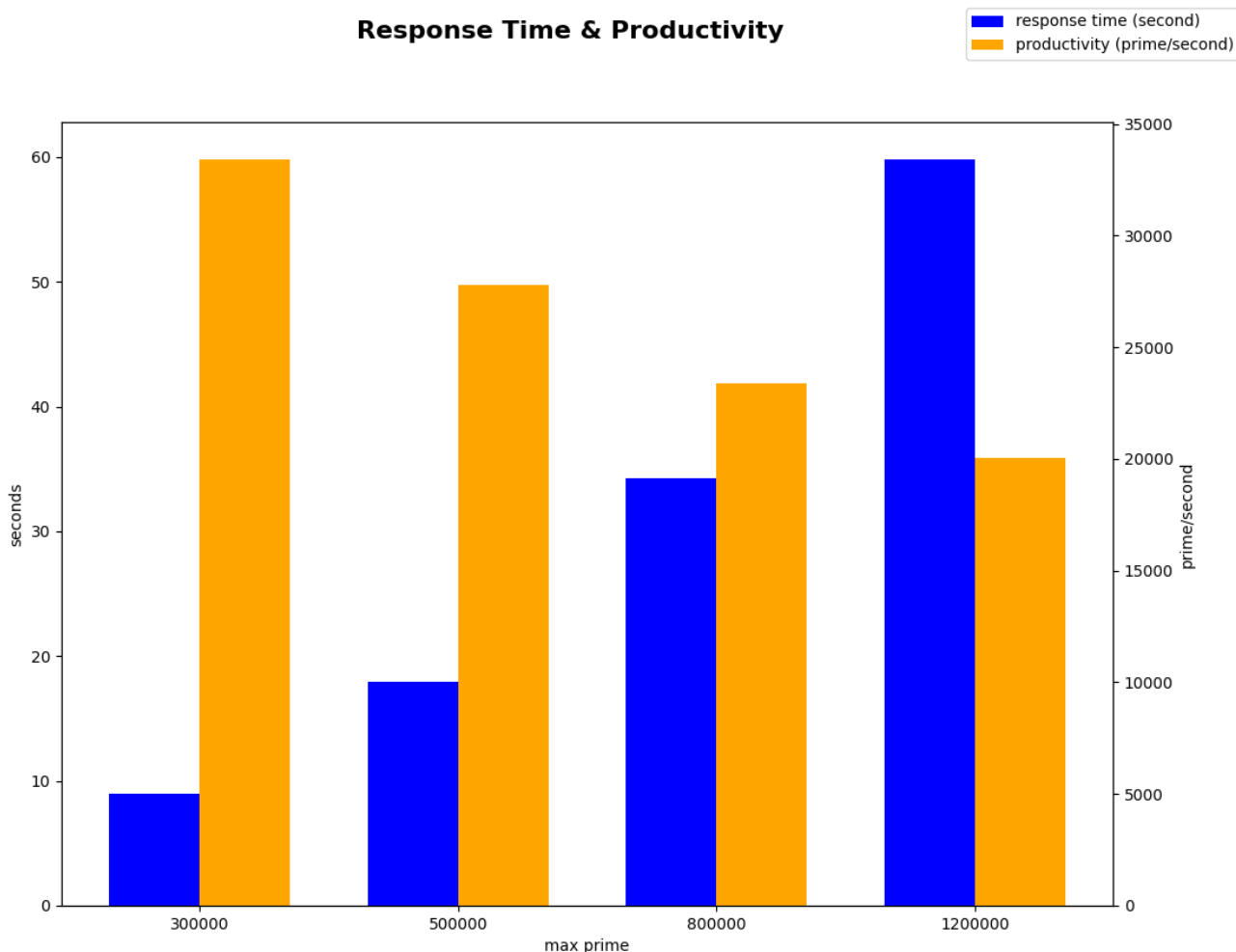
Se han ejecutado unas cargas de 300000, 500000, 800000 y 1200000 con el 75% de recursos de CPU ($12 \cdot 0.75 = 9 \text{ núcleos}$). Los tiempos de respuesta obtenidos para cada ejecución son los siguientes (en segundos):

N ejecución	300000	500000	800000	1200000
1	8.9768	17.9778	34.2206	59.7978
2	8.9731	17.9912	34.2708	59.8142
3	8.9805	17.9847	34.2054	59.8004
4	8.9901		34.2293	
5	8.9764		34.3251	
6	8.9826		34.2106	
7			34.2182	
8			34.2113	
9			34.2135	
Media	8.9799	17.9845	34.2338	59.8041

De los cuales se puede calcular la productividad (primos/segundo):

N ejecución	300000	500000	800000	1200000
1	33419.4813	27812.0793	23377.7315	20067.6279
2	33433.2616	27791.3646	23343.4877	20062.1257
3	33405.7123	27801.4089	23388.1200	20066.7554
4	33370.0403		23371.7896	
5	33420.9705		23306.5599	
6	33397.9026		23384.5650	
7			23379.3712	
8			23384.0865	
9			23382.5828	
Media	33407.8948	27801.6176	23368.6993	20065.5030

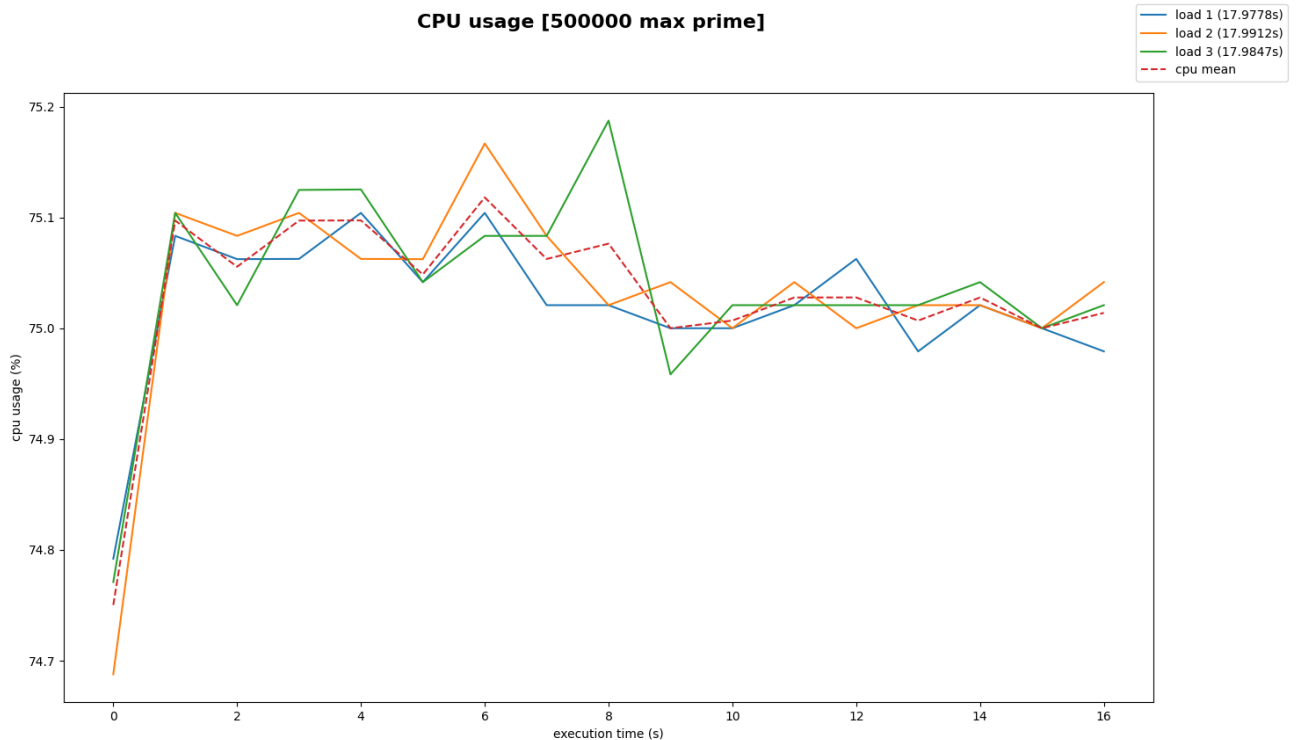
Podemos representar los valores medios en una gráfica para facilitar la interpretación del comportamiento:



Como se puede observar, el tiempo de respuesta incrementa de forma exponencial a medida que se va aumentando la carga. En cambio, la productividad tiene un comportamiento inverso al tiempo de respuesta. Como la productividad es $P = \frac{\text{carga}}{\text{tiempo respuesta}}$, esto significa que tiempo de respuesta crece más rápido que las diferentes cargas que hemos aplicado.

3.1 Utilización de CPU y de RAM

Cada ejecución de cada carga ha generado una serie de valores de CPU y de RAM (posiblemente cada serie tenga más o menos valores, dependiendo del tiempo que ha tardado cada ejecución). Por ejemplo, visto gráficamente, la utilización de la CPU de la ejecución de 500000 números primos es la siguiente:



Primero se ha calculado la media de cada instante de monitorización y después la media de todos estos valores. La siguiente tabla muestra el resultado de esta operación:

Carga (max prime)	Utilización CPU media (%)	Utilización RAM media (%)
300000	75.0542	6.7092
500000	75.0302	6.7148
800000	75.0305	6.7016
1200000	75.0272	6.7067

La utilización de la CPU se mantiene constante al 75% debido a la naturaleza del experimento. El ordenador tiene 12 núcleos y se le ha indicado al Sysbench que ponga 9 núcleos (el 75% del total de núcleos; $12 \cdot 0.75 = 9$) a trabajar al máximo con el parámetro `--num-threads`. Se puede observar este comportamiento con el monitor `htop`:

```

0[|||||100.0%] 3[|||||0.7%] 6[|||||100.0%] 9[|||||2.6%]
1[|||||100.0%] 4[|||||100.0%] 7[|||||100.0%] 10[|||||100.0%]
2[|||||100.0%] 5[|||||100.0%] 8[|||||0.7%] 11[|||||100.0%]
Mem[|||||1.00G/19.5G] Tasks: 117, 441 thr, 259 kthr; 11 running
Swp[|||||0K/6.40G] Load average: 2.04 0.47 0.15
Uptime: 07:11:44

Main I/O
PID USER PRI NI VIRT RES SHR S CPU%MEM% TIME+ Command
8481 hexd hog 20 0 77836 1664 1536 R 100.1 0.0 0:16.55 sysbench --num-threads 9 --test cpu --cpu-max-prime 1500000 run
8482 hexd hog 20 0 77836 1664 1536 R 100.1 0.0 0:16.55 sysbench --num-threads 9 --test cpu --cpu-max-prime 1500000 run
8483 hexd hog 20 0 77836 1664 1536 R 100.1 0.0 0:16.55 sysbench --num-threads 9 --test cpu --cpu-max-prime 1500000 run
8484 hexd hog 20 0 77836 1664 1536 R 100.1 0.0 0:16.55 sysbench --num-threads 9 --test cpu --cpu-max-prime 1500000 run
8476 hexd hog 20 0 77836 1664 1536 R 99.4 0.0 0:16.54 sysbench --num-threads 9 --test cpu --cpu-max-prime 1500000 run
8477 hexd hog 20 0 77836 1664 1536 R 99.4 0.0 0:16.53 sysbench --num-threads 9 --test cpu --cpu-max-prime 1500000 run
8478 hexd hog 20 0 77836 1664 1536 R 99.4 0.0 0:16.54 sysbench --num-threads 9 --test cpu --cpu-max-prime 1500000 run
8479 hexd hog 20 0 77836 1664 1536 R 99.4 0.0 0:16.54 sysbench --num-threads 9 --test cpu --cpu-max-prime 1500000 run
8480 hexd hog 20 0 77836 1664 1536 R 99.4 0.0 0:16.53 sysbench --num-threads 9 --test cpu --cpu-max-prime 1500000 run
8457 hexd hog 20 0 9236 5248 3200 R 2.0 0.0 0:00.52 htop

```

Debido a que 9 núcleos están al 100% y 3 tienen un porcentaje de carga despreciable (≈ 0), la utilización global de la CPU, U_{CPU} , se puede calcular de la siguiente forma:

$$U_{CPU} = \frac{100 \cdot 9 + 0 \cdot 3}{12} = 0.75 \Rightarrow 75\%$$

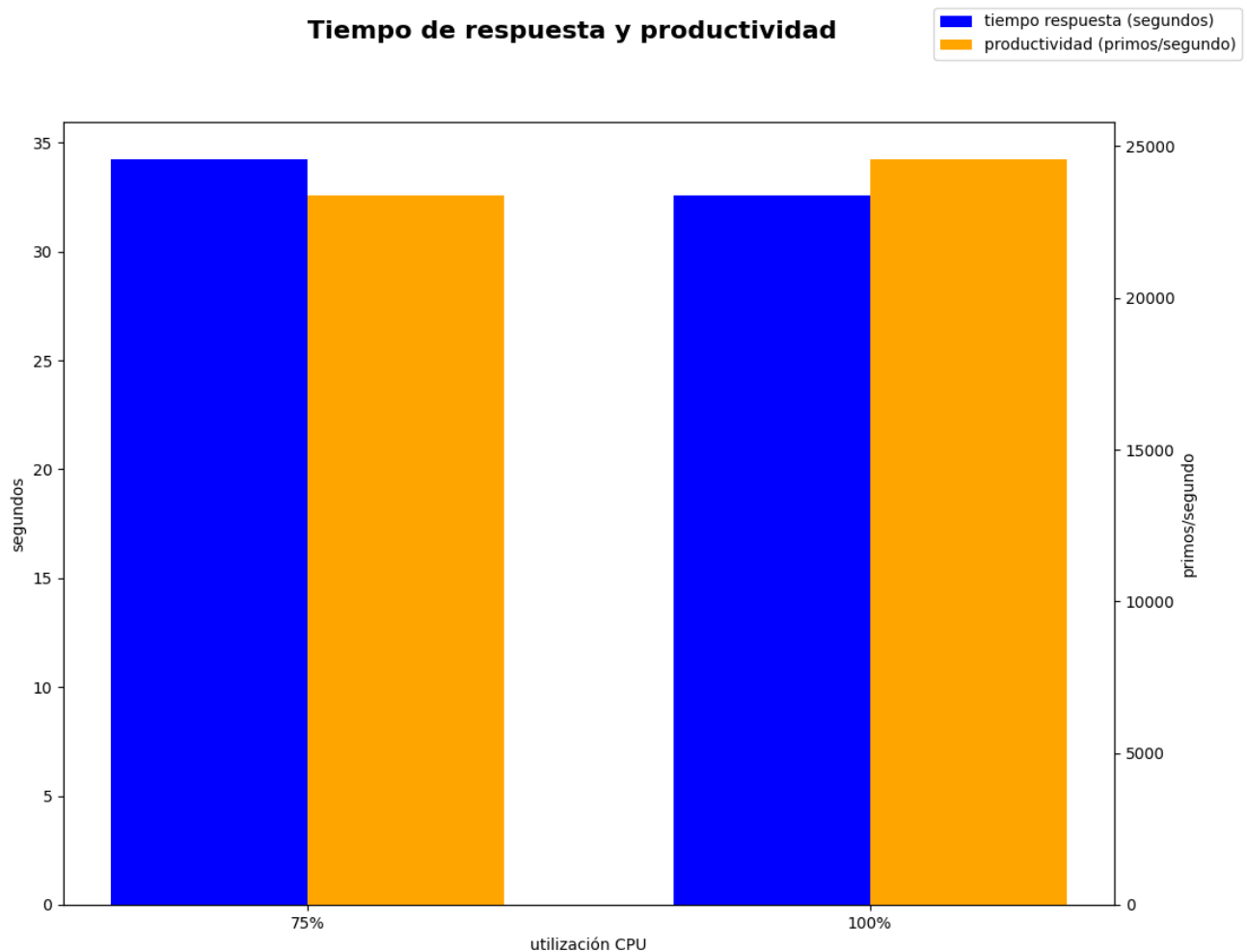
La utilización media de la RAM también se mantiene constante, de hecho si medimos la utilización antes de ejecutar Sysbench podemos ver que la utilización de memoria no cambia, debido a que el experimento está enfocado a la CPU y utiliza una cantidad mínima de memoria (la misma independientemente de la carga). Mi suposición es que Sysbench sólo reserva memoria para poder realizar los cálculos de números primos y descarta todos los resultados.

3.2 Diferentes utilizaciones de CPU

Se ha ejecutado el experimento con una carga de 800000 números primos pero con porcentajes de utilización total de CPU diferentes: uno al 75% (9 núcleos) y otro al 100% (12 núcleos).

Utilización CPU (%)	Tiempo Respuesta (s)	Productividad (primo/segundo)
75%	34.2338	23368.6993
100%	32.5639	24567.0334

Que representado gráficamente queda de la siguiente forma:



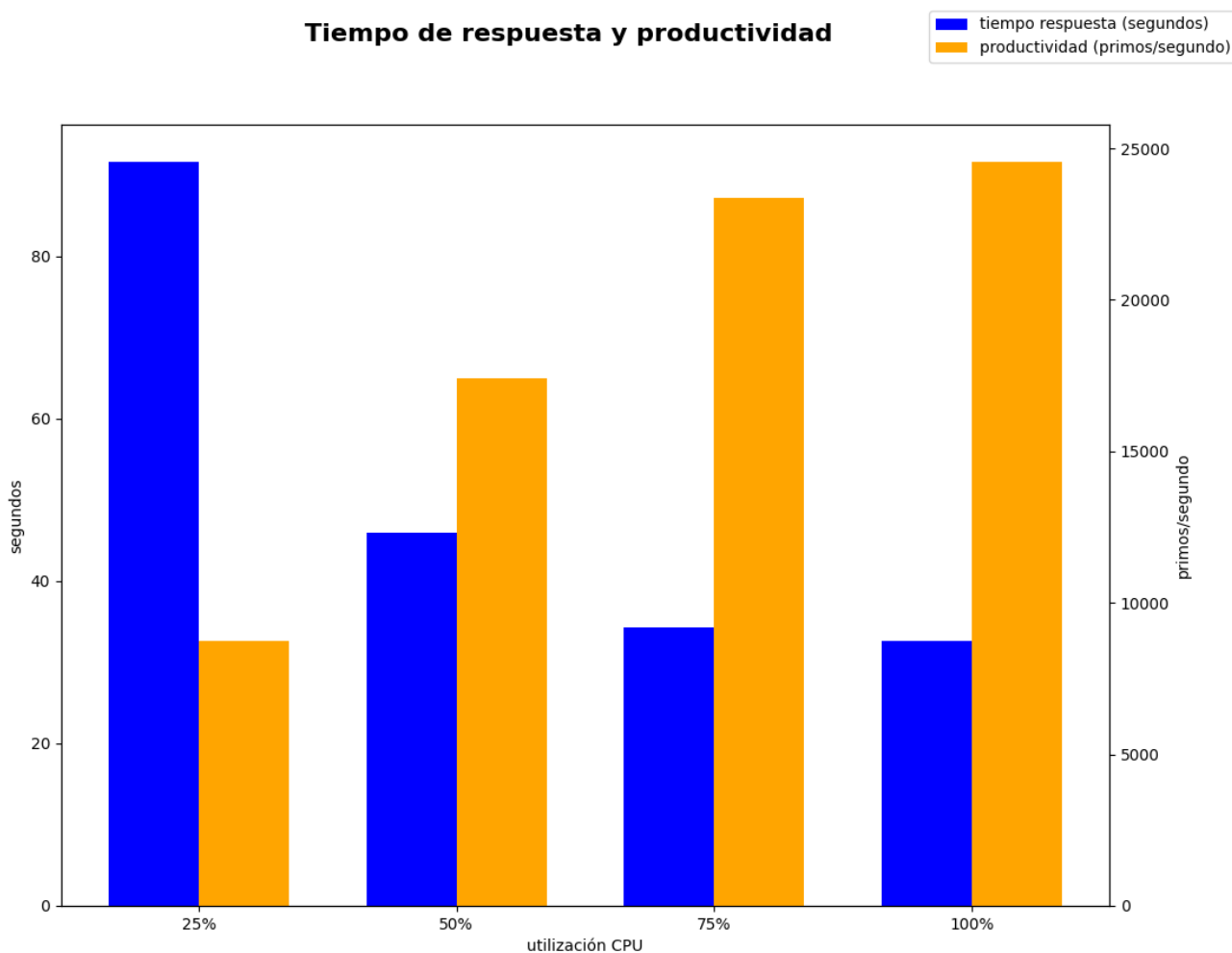
Como era de esperar, el experimento con el 100% de utilización total de CPU se ha ejecutado más rápido, casi 2 segundos, que el experimento con el 75%. Debido a que la carga es la misma en ambos casos, esto resulta en una productividad mayor en el caso del 100% de CPU.

3.3 Comportamiento del sistema con carga fija

Se ha fijado la carga a 800000 números primos y se ha variado el porcentaje de utilización total de CPU en 25% (3 núcleos), 50% (6 núcleos), 75% (9 núcleos) y 100% (12 núcleos).

Utilización CPU (%)	Tiempo Respuesta (s)	Productividad (primo/segundo)
25%	91.6455	8729.2862
50%	45.9799	17398.9168
75%	34.2348	23368.0214
100%	32.5799	24554.9603

Que representado gráficamente queda de la siguiente forma:



Si calculamos la aceleración del tiempo de respuesta, $R_{i,j} = \frac{R_i}{R_j}$ con i igual al numero de columna y j al numero de fila, para todas las combinaciones de utilizaciones de CPU, obtenemos la siguiente matriz:

	25%	50%	75%	100%
25%	1.0	0.5017	0.3735	0.3554
50%	1.9931	1.0	0.7445	0.7085
75%	2.6769	1.3430	1.0	0.9516
100%	2.8129	1.4112	1.0507	1.0

Basta con fijarse en la matriz triangular inferior porque tiene todas las relaciones con el tiempo de respuesta mayor en el numerador y es más cómodo hacer comparaciones (valores más altos $\implies$ más aceleración). Como se ha utilizado un 25% de incremento para cada rango de utilización, lo interesante es mirar la aceleración entre cada incremento (los valores justo debajo de la diagonal de la matriz). Para el primer incremento, del 25% al 50%, hay una aceleración de un $99.31\% = (1.9931 - 1) \cdot 100$, para la segunda, del 50% al 75%, un 34.30%, y la tercera, del 75% al 100%, solo un 5.07%.

Esto indica que el tiempo de respuesta cae, y la productividad crece (porque tiene una relación inversa al tiempo de respuesta), logarítmicamente a medida que los recursos de la CPU aumentan.

Otra característica que podemos ver es que rango es el mejor en términos del tiempo de respuesta según los recursos asignados. Para ello podemos multiplicar el tiempo de respuesta por el porcentaje de CPU representado como fracción y mirar que rango tiene el valor menor (menos tiempo de respuesta y menos recursos asignados):

Utilización CPU (%)	Tiempo Respuesta (s)	Tiempo Respuesta * % CPU
25%	91.6455	22.9113
50%	45.9799	22.9899
75%	34.2348	25.6761
100%	32.5799	32.5799

Podemos ver que en el caso del 25%, aunque es el que tiene el tiempo de respuesta más alto, tiene el mejor tiempo de respuesta por recurso de CPU. A medida que el porcentaje de recursos asignados aumenta el tiempo de respuesta por recurso de CPU empeora, lo que indica que la paralelización tiene un coste.